SELECT NEXT SECTION TO CONTINUE

Section	Questions	Time Limit	Status
Backend	7	40 m	Begin
DSA	4	60 m	Done

SUBMIT ASSESSMENT

SECTION DSA - QUESTIONS LIST

Q1	Permutation of the String	EDIT >
Q2	Airport Runways	EDIT >
Q3	Majority Element	EDIT >
Q4	Allocate Minimum Number of Pages	EDIT >

Coding Question

Mark For Review

Problem Statement:

You are given two strings **s1** and **s2** consisting of lowercase characters. You need to return **true**, if s2 contains a permutation of s1, or **false** otherwise. In other words, return true if one of s1's permutations is the substring of s2.

The function **isPermutation()** accepts the parameters string **s1** and **s2**. Complete the function **isPermutation()** and returns the **check_string** in boolean format.

For **Example**: If the string **s1** is tac and **s2** is catering then one of the permutation of **tac** i.e. **cat** is present in the **cat**ering, Hence return **true**.

Constraints:

1<=s1.size,s2.size<=1000

Example 1:

Input:

s1=qwd

s2= wdfijewjewewswf

Output:

check_string= False

Example 2:

Input:

s1=qwa

s1=qwa

s2= wdfijewjewewswf

Output:

check_string= False

Example 2:

Input:

s1=qwa

s2= wqawsdfgt

Output:

check_string= True

The example mentioned above is a **sample test case** for your understanding. The program will be tested on other **secret test cases** in the backend. Make sure you click the **SAVE AND NEXT** button to save and submit your answer.

14px | Dark Theme | JavaScript | Run | Reset

```
    // Initialize the frequency counts for the first window in s2
    for (let i = 0; i < str1.length; i++) {
        charCountWindow[str2.charCodeAt(i) - aCharCode]++;
    }

    // Initial comparison of the first window
    if (arraysEqual(charCountS1, charCountWindow)) {
        return true;
    }

    // Sliding the window over s2 to check other substrings
```

Save and Next

Coding Question

Mark For Review

Problem Statement

Dubai Airport is one of the busiest in the world. Given the arrival and departure times of all planes that pass through Dubai, your task is to find the minimum number of runways required to ensure that no plane has to circle the airport waiting for a runway to become available.

You need to implement the function `runways_required()`, which takes the following parameters:

- `arrival_time[]` : An array of integers representing the arrival times of the planes.
- `departure_time[]` : An array of integers representing the departure times of the planes.
- `no_of_planes` : An integer representing the total number of planes.

The function should return the minimum number of runways required as an integer.

Note

- The times are given in a 24-hour format (e.g., `hhmm`).

Constraints:

- `1 <= N <= 5000` (where `N` is the number of planes)

Example 1:

Sample Input:

arrival_time = [2200, 2300]
departure_time = [0200, 0300]
no_of_planes = 2

Sample Output:

Previous

Save and Next

- `1 <= N <= 5000` (where `N` is the number of planes)

Example 1:

Sample Input:

arrival_time = [2200, 2300]
departure_time = [0200, 0300]
no_of_planes = 2

Sample Output:

no_of_runways = 2

Explanation:

- The first flight arrives at 22:00 (10:00 PM) and is scheduled to depart at 02:00 AM.
- The second flight arrives at 23:00 (11:00 PM) and is scheduled to depart at 03:00 AM.
- Since the second flight arrives before the first flight departs, you need 2 runways.

Example 2:

Sample Input:

arrival_time = [0900, 0920, 0950, 1000, 1100, 1800]
departure_time = [0930, 1200, 1120, 1130, 1900, 2000]
no_of_planes = 6

Sample Output:

no_of_runways = 4

Explanation:

- Multiple flights arrive close to each other, and some flights are still on the runway when others arrive.
- The maximum overlap occurs when 4 flights are at the airport at the same time, so 4 runways are needed.

Coding Question

Mark For Review

Problem Statement:

You are given an array **arr** of size **n**. You need to find the majority element in an array. A majority element in an array **arr** of size **n** is an element that appears strictly greater than n/2 times in the array. If no majority exists, return **-1**.

The function **major_element()** accepts the parameters array of integer **arr** having size **n** .Complete the function **major_element()** and return the value of **maximum_times** in the integer format.

For **Example**: If the array **arr** is [2,2,3,4,2,3,2,2] of size **8** the number of occurrences of 2 is **5** which is more than (8/2=4).Hence majority element will be 2.

Example 1:

Input:

n=10

arr= [6,1,6,2,6,6,5,24,6,22]

Output:

maximum_times= -1

Example 2:

Input:

n=9

Previous

Save and Next

Example 2:

Input:

n=9

arr=[1,2,3,3,3,3,5,6,3]

Output:

maximum_times= 3

The example mentioned above is a **sample test case** for your understanding. The program will be tested on other **secret test cases** in the backend. Make sure you click the **SAVE AND NEXT** button to save and submit your answer.

14px | Dark Theme | JavaScript | Run | Reset

```
function major_element(arr, n)
 {
   var maximum_times=0;
   //let candidate = null;
   let count = 0;
   for (let num of arr) {
       if (count === 0) {
           maximum_times = num;
       }
       count += (num === maximum_times)
           ? 1 : -1;
   }


  //Write your code here without removing the existing code
   //the variable 'arr' contains array of integers of size n.
   //modified the variable 'maximum_times' contain the output of the program in integer format
```

Previous | Save and Next

Coding Question

Mark For Review

Problem Statement:

You are given **n** number of books. Every **ith** book has **arr[i]** number of pages. You have to allocate contiguous books to **k** number of students. There can be many ways or permutations to do so. In each permutation, one of the k students will be allocated the maximum number of pages. Out of all these permutations, the task is to find that particular permutation in which the **maximum** number of pages allocated to a student is the minimum of those in all the other permutations and print this minimum value.Each book will be allocated to exactly one student. Each student has to be allocated at least one book.

Note: Return -**1** if a valid assignment is not possible, and allotment should be in contiguous order.

The function **minimum_pages()** accepts the parameters array of integer **arr** with the size **n** and an integer **k**. Complete the function **minimum_pages()** and return the **allocated_value** in the integer format.

For **Example**: If the **arr** is [12,34,67,90] of size **n** and **k** is 2 then Allocation can be done in following ways:{12} and {34, 67, 90} ,Maximum Pages = 191{12, 34} and {67, 90} ,Maximum Pages = 157{12, 34, 67} and {90} ,Maximum Pages =113. Therefore, the minimum of these cases is 113, which is selected as the output.

Constraints:-

1<=n,k<=1000

Example 1:

Input:

n=8

Previous

Save and Next

Example 1:

Input:

n=8

arr= [11,33,15,62,22,15,31,60]

k=3

Output:

allocated_value = 99

Example 2:

Input:

n=2

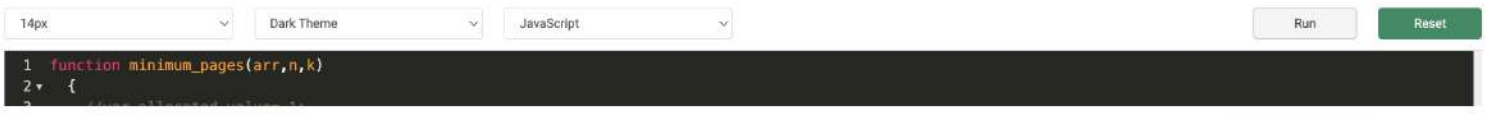
arr= [12,3]

k=2

Output:

allocated_value= 12

The example mentioned above is a **sample test case** for your understanding. The program will be tested on other **secret test cases** in the backend. Make sure you click the **SAVE AND NEXT** button to save and submit your answer.



Previous

Save and Next

Q1	Unique CustomerIDs	EDIT >
Q2	Leads in a Month	EDIT >
Q3	Employees Management	EDIT >
Q4	Highest Salary	EDIT >
Q5	Average age of customers	EDIT >
Q6	Employee Info Even	EDIT >
Q7	Employees' Payroll	EDIT >

have the same structure: OrderID, CustomerID, and Amount.

Task: You need to retrieve a list of unique CustomerIDs along with the total amount spent by each customer. However, if a customer has both orders and sales, you want to combine the amounts into a single total.

Table name: Orders

Table Structure

Column Name	Type
OrderID	INT
CustomerID	INT
Amount	DECIMAL

**Assume no field in the table have null values.

Table name: Sales

Table Structure

Column Name	Type
SaleID	INT
CustomerID	INT
Amount	DECIMAL

```
FROM (
    SELECT CustomerID, Amount FROM Orders
    UNION ALL
    SELECT CustomerID, Amount FROM Sales
) AS Combined
GROUP BY CustomerID;
```

Run

Save and Next

**Assume no field in the table have null values.

Table name: Sales

Table Structure

Column Name	Type
SaleID	INT
CustomerID	INT
Amount	DECIMAL

**Assume no field in the table have null values.

Answer Structure

Query the tables to print the unique CustomerIDs along with their total spending, combining both orders and sales amounts. The output must have 2 columns.

Sample Output

CustomerID	TotalSpending
101	100
102	110
103	60

```
3  FROM (
4      SELECT CustomerID, Amount FROM Orders
5      UNION ALL
6      SELECT CustomerID, Amount FROM Sales
7  ) AS Combined
8  GROUP BY CustomerID;
9
```

Run

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

Use the following table and write the SQL query in the editor below.

Task: You need not create the table or insert any data. Write an SQL Query to return the count of the total number of leads in the months of APRIL, MAY and JUNE

Table Name: Leads

Table Structure:

- Lead_Id (INT)
- Created_Date (Varchar)
- Industry (Varchar)

Answer Structure:
Query the *Leads* table to complete the task. The output must have just 1 column.

Sample Input

Lead_ID	Created_Date	Industry
101	FEBRUARY	HR
111	MAY	Health

```
1  -- ENTER YOUR SQL STATEMENTS HERE
2
3  SELECT COUNT(*) AS lead_count
4  FROM Leads
5  WHERE (Created_Date) IN ('April', 'May',
   'June');
6
7
```

Previous

Save and Next

months of APRIL, MAY and JUNE

Table Name: Leads

Table Structure:

- Lead_Id (INT)
- Created_Date (Varchar)
- Industry (Varchar)

Answer Structure:
Query the *Leads* table to complete the task. The output must have just 1 column.

Sample Input

Lead_ID	Created_Date	Industry
101	FEBRUARY	HR
111	MAY	Health
104	JUNE	Pharma

Sample Output

COUNT (Created_Date)
2

```
4  FROM Leads
5  WHERE (Created_Date) IN ('April', 'May',
   'June');
6
7
```

Run

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

You are given three tables: Employees, Departments, and Managers.

Task: Retrieve a list of employees and their corresponding department names along with the name of their respective managers. If an employee does not have a manager, display "No Manager" in the manager's name column.

Table name: employees

Table Structure

Column Name	Type
emp_id	INT
emp_name	VARCHAR(50)
department_id	INT
manager_id	INT

Table name: departments

Table Structure

Column Name	Type
dept_id	INT
department_name	VARCHAR(50)

**Assume no field in the table have null values.

Table name: managers

Table Structure

```sql
-- ENTER YOUR SQL STATEMENTS HERE

SELECT d.department_name, AVG(s.salary) AS
average_salary
FROM salaries s
JOIN employees e ON s.emp_id = e.emp_id
JOIN departments d ON e.department_id =
d.dept_id
GROUP BY d.department_name
ORDER BY average_salary DESC
LIMIT 5;
```

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

Table Structure

Column Name	Type
dept_id	INT
department_name	VARCHAR(50)

**Assume no field in the table have null values.

Table name: managers

Table Structure

Column Name	Type
manager_id	INT
manager_name	VARCHAR(50)

**Assume no field in the table have null values.

Answer Structure

Query the tables to print employees, their respective department and managers. The output must have 3 columns.

Sample Output

employee_name	department_name	manager_name
John Smith	HR	No Manager
Jane Doe	Finance	Emily Johnson
Michael Lee	IT	Emily Johnson

```sql
-- ENTER YOUR SQL STATEMENTS HERE

SELECT d.department_name, AVG(s.salary) AS
average_salary
FROM salaries s
JOIN employees e ON s.emp_id = e.emp_id
JOIN departments d ON e.department_id =
d.dept_id
GROUP BY d.department_name
ORDER BY average_salary DESC
LIMIT 5;
```

Previous

Save and Next

ID. The "Departments" table contains the department ID and their corresponding department names and the "Salaries" table contains employee ID and salaries.

Task: Write a SQL query to find the highest paid employee in each department.

Table name: Employees

Table Structure

Column Name	Type
EmpID	INT
EmpName	VARCHAR(50)
DeptID	INT

**Assume no field in the table have null values.

Table name: Departments

Table Structure

Column Name	Type
DeptID	INT
DeptName	VARCHAR(50)

**Assume no field in the table have null values.

```sql
SELECT d.department_name, AVG(s.salary) AS
average_salary
FROM salaries s
JOIN employees e ON s.emp_id = e.emp_id
JOIN departments d ON e.department_id =
d.dept_id
GROUP BY d.department_name
ORDER BY average_salary DESC
LIMIT 5;
```

Run

Previous

Save and Next

DeptID	INT
DeptName	VARCHAR(50)

**Assume no field in the table have null values.

Table name: Salaries

Table Structure

Column Name	Type
EmpID	INT
Salary	DECIMAL

**Assume no field in the table have null values.

Answer Structure

Query the tables to print the highest paid employee in each department. The output must have 3 columns.

Sample Output

DeptName	EmpName	Salary
Sales	John Doe	60000
Marketing	Bob Johnson	62000
Finance	Michael Clark	70000

```
4  SELECT d.department_name, AVG(s.salary) AS
   average_salary
5  FROM salaries s
6  JOIN employees e ON s.emp_id = e.emp_id
7  JOIN departments d ON e.department_id =
   d.dept_id
8  GROUP BY d.department_name
9  ORDER BY average_salary DESC
10 LIMIT 5;
```

Run

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

Use the following table and write the SQL query in the editor below.

Task: Write a SQL query to list the even EmpIDs from the table.

Table Name: *EMPLOYEEINFO*

Table Structure

Column Name	Type
EmpID	INT **(Primary Key)**
EmpFname	VARCHAR (50)
EmpLname	VARCHAR (50)
Department	VARCHAR (50)
Project	VARCHAR (50)
Address	VARCHAR (100)
DOB	DATE
Gender	CHAR (1)

**Assume that no field in the table has a null value

Answer Structure

```
1  -- ENTER YOUR SQL STATEMENTS HERE
```

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

DOB	DATE
Gender	CHAR (1)

**Assume that no field in the table has a null value

Answer Structure

Query the table *EMPLOYEEINFO* to return the list of even EmpIDs. The output must have just 1 column

Sample Input

EmpID	EmpFname	EmpLname	Department	Project	Location	DOB	Gender
1	Sant	Mehra	HR	P1	Hyderabad(HYD)	1976-01-12	M
2	Ananya	Mishra	Admin	P2	Delhi(DEL)	1968-02-05	F
3	Rohan	Diwan	Account	P3	Mumbai(BOM)	1980-01-01	M
4	Sonia	Kulkarni	HR	P1	Hyderabad(HYD)	1992-02-05	F
5	Ankit	Kapoor	Admin	P2	Delhi(DEL)	1994-03-07	M

Sample Output

```
1  -- ENTER YOUR SQL STATEMENTS HERE
```

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

Answer Structure

Query the table *EMPLOYEEINFO* to return the list of even EmpIDs. The output must have just 1 column

Sample Input

EmpID	EmpFname	EmpLname	Department	Project	Location	DOB	Gender
1	Sant	Mehra	HR	P1	Hyderabad(HYD)	1976-01-12	M
2	Ananya	Mishra	Admin	P2	Delhi(DEL)	1968-02-05	F
3	Rohan	Diwan	Account	P3	Mumbai(BOM)	1980-01-01	M
4	Sonia	Kulkarni	HR	P1	Hyderabad(HYD)	1992-02-05	F
5	Ankit	Kapoor	Admin	P2	Delhi(DEL)	1994-03-07	M

Sample Output

EmpID
2
4

```
1  -- ENTER YOUR SQL STATEMENTS HERE
```

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

Use the following table and write the SQL query in the editor below.

Task: Write an SQL query to find the average age of customers whose email addresses end with ".com", and display it along with the total number of such customers.

Table Name: CUSTOMERS

Table Structure

- ID (INT) (*Primary Key*)
- NAME (VARCHAR)
- AGE (VARCHAR)
- EMAIL (INT)

Answer Structure

Query the *CUSTOMERS* table to complete the task. The output is case-sensitive. It must consist of 2 columns with the following names:

- *NUM_CUSTOMERS* - Use the column to indicate the total no.of. customers.
- *AVG_AGE* - Use the column to indicate the average age of customers,

Sample Input

ID	NAME	AGE	EMAIL

```
1  -- ENTER YOUR SQL STATEMENTS HERE
2  SELECT COUNT(*) from
```

Previous

Save and Next

Coding Question

Mark For Review

14px | Dark Theme | Run

- EMAIL (INT)

Answer Structure

Query the *CUSTOMERS* table to complete the task. The output is case-sensitive. It must consist of 2 columns with the following names:

- *NUM_CUSTOMERS* - Use the column to indicate the total no.of. customers.
- *AVG_AGE* - Use the column to indicate the average age of customers,

Sample Input

ID	NAME	AGE	EMAIL
1	John	25	john@example.io
2	Jane	35	jane@example.com
3	Bob	42	bob@spreadsheet.org
4	Alice	29	alice@example.co
5	Michael	22	michael@visbo.com

Sample Output

NUM_CUSTOMERS	AVG_AGE
2	28.5

```
1  -- ENTER YOUR SQL STATEMENTS HERE
2  SELECT COUNT(*) from
```

Previous

Save and Next

Table Name: *EMPLOYEEINFO*

Table Structure

Column Name	Type
EmpID	INT **(Primary Key)**
EmpFname	VARCHAR (50)
EmpLname	VARCHAR (50)
Department	VARCHAR (50)
Project	VARCHAR (50)
Address	VARCHAR (100)
DOB	DATE
Gender	CHAR (1)

**Assume that no field in the table has a null value

Answer Structure

Query the table *EMPLOYEEINFO* to return the list of even EmpIDs. The output must have just 1 column

Run

Previous

Save and Next

Sample Input

EmpID	EmpFname	EmpLname	Department	Project	Location	DOB	Gender
1	Sant	Mehra	HR	P1	Hyderabad(HYD)	1976-01-12	M
2	Ananya	Mishra	Admin	P2	Delhi(DEL)	1968-02-05	F
3	Rohan	Diwan	Account	P3	Mumbai(BOM)	1980-01-01	M
4	Sonia	Kulkarni	HR	P1	Hyderabad(HYD)	1992-02-05	F
5	Ankit	Kapoor	Admin	P2	Delhi(DEL)	1994-03-07	M

Sample Output

EmpID
2
4

Run

Previous

Save and Next